

WEEK 3 TASK: DEBUGGING AND PERFORMANCE OPTIMIZATION

Real Hardware / Simulator Setup – ESP32 + DHT22 Temperature Monitoring

21. Real Hardware / Simulator Setup

To make the debugging and optimization exercise representative of an actual embedded-system project, the firmware is implemented for an ESP32 development board with a DHT22 temperature sensor, an LED indicator, and a USB serial-debug connection. The same circuit can be reproduced in a simulator such as Wokwi when physical hardware is unavailable. The setup provides a realistic sensor input, GPIO control, serial logging, breakpoints, and timing measurements.

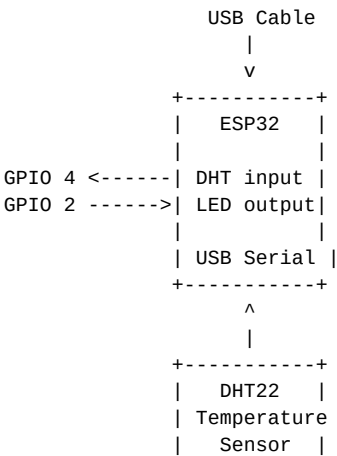
21.1 Hardware Components

Component	Quantity	Purpose
ESP32 development board	1	Microcontroller and USB serial interface
DHT22 temperature sensor	1	Temperature measurement
LED	1	High-temperature warning indicator
220 Ω resistor	1	LED current limiting
10 k Ω resistor	1	DHT22 data-line pull-up, if required by the module
Breadboard	1	Circuit assembly
Jumper wires	Several	Electrical connections
USB cable	1	Power, programming, and serial monitoring

21.2 Pin Connections

Device	Pin	ESP32 Connection
DHT22	VCC	3.3 V
DHT22	GND	GND
DHT22	DATA	GPIO 4
LED	Anode (+)	GPIO 2 through 220 Ω resistor
LED	Cathode (-)	GND

21.3 Hardware Block Diagram



+-----+

```
GPIO 2 --> 220 ohm --> LED --> GND
DHT22 DATA --> GPIO 4
DHT22 VCC --> 3.3 V
DHT22 GND --> GND
```

21.4 Simulator Alternative

If physical hardware is not available, the same experiment can be reproduced in a browser-based ESP32 simulator. Create an ESP32 project, add a DHT22 sensor and an LED, connect the components according to the pin table above, compile the firmware, start the simulation, and open the serial monitor. The simulated temperature can then be changed to values such as 30°C, 49°C, 50°C, 55°C, and 75°C to reproduce the debugging test cases.

21.5 Firmware Test Setup

Test sequence:

1. Power/program the ESP32.
2. Start the serial monitor at the firmware's configured baud rate.
3. Read the DHT22 temperature.
4. Compare the reading with `TEMP_THRESHOLD = 50°C`.
5. Keep LED OFF below 50°C.
6. Turn LED ON at 50°C or above.
7. Record serial output and timing.
8. Repeat for boundary and abnormal values.

21.6 Practical Debugging Procedure

The debugger or simulator is used to pause execution at the temperature comparison and inspect the sensor variable. A breakpoint is placed on the condition ``if (temperature >= TEMP_THRESHOLD)``. The watch window is used to inspect temperature and threshold values. Serial logging is used to correlate the observed sensor value with the LED state. For performance measurements, a GPIO can be toggled immediately before and after the section under test; the pulse width can then be measured with a logic analyzer or oscilloscope. In a simulator, the serial log and simulator timing facilities can be used as supporting evidence.

21.7 Optimized ESP32-Style Firmware

```
#include <Arduino.h>
#include <DHT.h>

#define DHT_PIN 4
#define DHT_TYPE DHT22
#define LED_PIN 2
#define TEMP_THRESHOLD 50.0f

DHT dht(DHT_PIN, DHT_TYPE);

void setup(void)
{
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);
    Serial.begin(115200);
    dht.begin();
}

void loop(void)
{
    float temperature = dht.readTemperature();

    if (isnan(temperature))
    {
        Serial.println("Sensor read error");
        digitalWrite(LED_PIN, LOW);
        delay(2000);
        return;
    }

    if (temperature >= TEMP_THRESHOLD)
    {
        digitalWrite(LED_PIN, HIGH);
        Serial.printf("High Temperature: %.1f C\n", temperature);
    }
}
```

```

    }
    else
    {
        digitalWrite(LED_PIN, LOW);
        Serial.printf("Temperature Normal: %.1f C\n", temperature);
    }

    delay(2000);
}

```

21.8 Expected Validation Results

Temperature	Expected LED	Expected Serial State
30°C	OFF	Temperature Normal
49°C	OFF	Temperature Normal
50°C	ON	High Temperature
55°C	ON	High Temperature
75°C	ON	High Temperature

21.9 Evidence to Capture

For submission, capture screenshots/photos of: (1) the assembled ESP32 circuit or simulator wiring, (2) the source code and build output, (3) the debugger breakpoint/watch window, (4) the serial monitor showing normal and high-temperature states, (5) the LED changing state, and (6) the timing measurement. These artifacts should be inserted into the final report with captions and dates where appropriate.

22. Updated Performance Validation Note

The earlier report used a simulated sensor value and qualitative performance comparison. With the ESP32/DHT22 setup, performance evidence can be made more rigorous by recording measured loop timing, serial-output frequency, and compiled flash/RAM statistics. The report should label measured values as actual measurements and should not substitute estimated numbers for hardware measurements. The key optimization remains the removal of the 100,000-iteration no-op loop and duplicate debug output.